

Teaching and Mentorship Statement

Emmanouil Giortamis
Technical University of Munich (TUM)
E-Mail: emmanouil.giortamis@tum.de

1 Teaching Philosophy and Experience

The opportunity to teach and mentor students is a primary reason I am pursuing a career in academia. Teaching deepens my own understanding of complex material by requiring me to explain intricate concepts clearly, and in quantum computing that clarity is essential both to attract new talent and to bridge the gap between high-level theory and physical hardware. I have been actively involved in teaching since my undergraduate studies. Beyond the classroom, I derive great satisfaction from guiding students as they take increasing ownership of their research and from witnessing their achievements.

My pedagogical approach balances solid theoretical foundations with practical implementation. This is especially critical in quantum computing, where students must move beyond quantum physics to understand fundamental software and systems challenges such as performance, scalability, and efficiency. My goal is to ensure students grasp these conceptual pillars while gaining the technical proficiency to build real systems: compilers, operating systems, and hardware-software co-designs. By integrating theoretical lectures with hands-on systems development, students internalize how high-level algorithms interact with the limitations of current and emerging quantum hardware. I also set ambitious expectations: when courses and research projects are intellectually rigorous, students consistently rise to meet them, uncovering capabilities they did not realize they possessed. My role is to supply the support, technical scaffolding, and strategic guidance that lets them flourish at that level, and I measure my success by their growth and achievements.

I have accumulated seven years of teaching experience, spanning fourteen semesters, at the Technical University of Munich (TUM) and the University of Crete, engaging with students at both undergraduate and graduate levels. My formal involvement began during my undergraduate studies at the University of Crete, where I served as a teaching assistant for one semester after receiving a merit-based undergraduate teaching assistantship awarded on overall academic standing. I taught for four further semesters during my Master's studies, and for nine semesters as a doctoral researcher at TUM.

In these roles I have moved from supporting positions to leadership ones: designing practical, hands-on exercises with automated grading infrastructure, leading student project teams, and organizing teaching assistant groups. I have also prepared course materials and served as lead instructor for laboratory courses and seminars. My core expertise is in computer systems: operating systems, cloud systems, distributed computing, and quantum software systems. The following table summarizes my teaching experience.

Course Title	Level	Description
Quantum Software Systems Seminar	UG/PG	Analysis, presentation, and reporting of research papers from top conferences.
Cloud Software Engineering Lab	UG/PG	Design and implementation of distributed cloud systems concepts and services.
Distributed Systems	UG/PG	Fundamental principles of distributed and cloud systems.
Languages and Compilers	UG	Fundamental programming language and compiler design principles.
Principles of Distributed Computing	UG/PG	Fundamental distributed systems principles.
Data Structures	UG	Data structures and algorithms.
Introduction to Computer Science	UG	Introductory programming using the C programming language.

UG = undergraduate, PG = graduate (Master's level).

2 Mentorship Experience

I have mentored 14 students: ten through thesis and independent study projects at the undergraduate and Master's levels, and four junior Ph.D. researchers whom I currently advise. Specifically, I have supervised eight theses (seven Master's and one Bachelor's) and two independent study projects, spanning scalable quantum cloud scheduling, hardware-aware circuit optimization, distributed quantum computing, fault-tolerant compilation, and real-time control planes for quantum error correction.

The outcome I am proudest of is that this mentorship does not end at the thesis defense. I work with students to carry their thesis results through to peer-reviewed publication, and in several cases through to a Ph.D. position in my group. I am a co-author on every paper listed below, having guided each project from its initial thesis scope through experimental design, writing, and submission.

Student	Mentorship began as	Resulting publications
Nathaniel Tornow	B.Sc. thesis → independent study	First author: QVM (PLDI '25), qTPU (OSDI '26)
Peter Wegmann	M.Sc. thesis → my Ph.D. student	First author: Chipmunq*, Zero-G*
Jannik Pflieger	M.Sc. thesis	First author: Harvest*; co-author: ECCentric (SIGMETRICS '26)
Dmitry Lugovoy	M.Sc. thesis	Co-author: Qonductor (SC '25, best student paper finalist)
Aleksandra Świerkowska	Ph.D. student	First author: ECCentric (SIGMETRICS '26)
Francisco Romão	Ph.D. student	Co-first author: Weaver (CGO '25); first author: MultiQ*, AtomGuard*
Anubhav Panda	Ph.D. student	Co-author: QShell*

* Under submission.

Two trajectories illustrate the aim. Nathaniel Tornow began with a Bachelor's thesis on distributed quantum circuit simulation, continued through an independent study project, and is now first author on papers

at PLDI and OSDI. Peter Wegmann began with a Master’s thesis on fault-tolerant compilation for chiplet architectures, is now my Ph.D. student, and leads two submissions of his own.

My approach to mentorship rests on three principles. First, students are most productive when genuinely excited about their work, so I make it a priority to communicate each project’s broader vision and real-world impact, and to align topics with student interests. Second, each student is unique and benefits from a personalized approach: some thrive with frequent collaborative problem-solving sessions, while others excel with greater independence to form their own paths. Third, I encourage students to take gradual ownership of their research. In a nascent field like quantum software systems, it is imperative to guide students from initial close collaboration to a state where they can independently test their own ideas. This incremental approach builds confidence and problem-solving skills, preparing them to lead future research.

3 Teaching Plans

My expertise in quantum software systems, distributed systems, and cloud software engineering positions me to teach core systems courses, including cloud and data systems, distributed systems, and quantum computing, at both undergraduate and graduate levels. I am equally comfortable teaching compilers, operating systems, and computer architecture, and I would bring a systems perspective to adjacent areas such as high-performance computing and quantum machine learning, where resource-management questions are badly under-taught relative to the algorithms.

My teaching philosophy prioritizes “learning by building”: students internalize theoretical concepts by implementing complex systems, such as a basic quantum compiler or a hybrid scheduler, from the ground up. Drawing on my research in quantum operating systems and hybrid cloud orchestration, I plan to offer specialized modules on hybrid resource estimation and orchestration that emphasize system-level efficiency in modern quantum clouds. I also intend to lead research seminars on recent results from venues such as ASPLOS, SC, MICRO, and ISCA, helping students track fast-moving work while refining their communication skills. Finally, I am committed to supervising doctoral students and to building a group in which junior researchers publish as first authors early. The table below presents my proposed courses.

Course Type	Planned Courses
Lectures	Introduction to Quantum Software Systems (UG) Principles of Scalable and Distributed Quantum Computing (PG)
Laboratory Courses	The Quantum System Stack Lab (UG/PG) Systems for Hybrid Quantum-Classical Computing (UG/PG) Cloud Software Engineering Lab (PG)
Seminars	Systems Research for Emerging Quantum Technologies (UG/PG) Hardware-Software Co-Design for Quantum Computing (PG) Distributed Systems, Orchestration, and Cloud Platforms (UG/PG)